

Chapter 7. Recognizing C Code Constructs in Assembly

In [Chapter 5](#), we reviewed the x86 architecture and its most common instructions. But successful reverse engineers do not evaluate each instruction individually unless they must. The process is just too tedious, and the instructions for an entire disassembled program can number in the thousands or even millions. As a malware analyst, you must be able to obtain a high-level picture of code functionality by analyzing instructions as groups, focusing on individual instructions only as needed. This skill takes time to develop.

Let's begin by thinking about how a malware author develops code to determine how to group instructions. Malware is typically developed using a high-level language, most commonly C. A code construct is a code abstraction level that defines a functional property but not the details of its implementation. Examples of code constructs include loops, `if` statements, linked lists, `switch` statements, and so on. Programs can be broken down into individual constructs that, when combined, implement the overall functionality of the program.

This chapter is designed to start you on your way with a discussion of more than ten different C code constructs. We'll examine each construct in assembly, although the purpose of this chapter is to assist you in doing the reverse: Your goal as a malware analyst will be to go from disassembly to high-level constructs. Learning in this reverse direction is often easier, because computer programmers are accustomed to reading and understanding source code.

This chapter will focus on how the most common and difficult constructs, such as loops and conditional statements, are compiled. After you've built a foundation with these, you'll learn how to develop a high-level picture of code functionality quickly.

In addition to discussing the different constructs, we'll also examine the differences between compilers, because compiler versions and settings can impact how a particular construct appears in disassembly. We'll evaluate two different ways that `switch` statements and function calls can be compiled using different compilers.

This chapter will dig fairly deeply into C code constructs, so the more you understand about C and programming in general, the more you'll get out of it. For help with the C language, have a look at the classic *The C Programming Language* by Brian Kernighan and Dennis Ritchie (Prentice-Hall, 1988). Most malware is written in C, although it is sometimes written in Delphi and C++. C is a simple language with a close relationship to assembly, so it is the most logical place for a new malware analyst to start.

As you read this chapter, remember that your goal is to understand the overall functionality of a program, not to analyze every single instruction. Keep this in mind, and don't get bogged down with the minutiae. Focus on the way programs work in general, not on how they do each particular thing.

Global vs. Local Variables

Global variables can be accessed and used by any function in a program. Local variables can be accessed only by the function in which they are defined. Both global and local variables are declared similarly in C, but they look completely different in assembly.

Following are two examples of C code for both global and local variables. Notice the subtle difference between the two. The global example, [Example 7-1](#), defines *x* and *y* variables outside the function. In the local example, [Example 7-2](#), the variables are defined within the function.

Example 7-1. A simple program with two global variables

```
int x = 1;
int y = 2;

void main()
{
    x = x+y;
    printf("Total = %d\n", x);
}
```

Example 7-2. A simple program with two local variables

```
void main()
{
    int x = 1;
    int y = 2;

    x = x+y;
    printf("Total = %d\n", x);
}
```

The difference between the global and local variables in these C code examples is small, and in this case the program result is the same. But the disassembly, shown in [Example 7-3](#) and [Example 7-4](#), is quite different. The global variables are referenced by memory addresses, and the local variables are referenced by the stack addresses.

In [Example 7-3](#), the global variable `x` is signified by `dword_40CF60`, a memory location at `0x40CF60`. Notice that `x` is changed in memory when `eax` is moved into `dword_40CF60` at **1**. All subsequent functions that utilize this variable will be impacted.

Example 7-3. Assembly code for the global variable example in [Example 7-1](#)

```
00401003      mov     eax, dword_40CF60
00401008      add     eax, dword_40C000
0040100E      mov     dword_40CF60, eax 1
00401013      mov     ecx, dword_40CF60
00401019      push    ecx
0040101A      push    offset aTotalD ; "total = %d\n"
0040101F      call    printf
```

In [Example 7-4](#) and [Example 7-5](#), the local variable `x` is located on the stack at a constant offset relative to `ebp`. In [Example 7-4](#), memory location `[ebp-4]` is used consistently throughout this function to reference the local variable `x`. This tells us that `ebp-4` is a stack-based local variable that is referenced only in the function in which it is defined.

Example 7-4. Assembly code for the local variable example in [Example 7-2](#), without labeling

```
00401006      mov     dword ptr [ebp-4], 0
0040100D      mov     dword ptr [ebp-8], 1
00401014      mov     eax, [ebp-4]
00401017      add     eax, [ebp-8]
0040101A      mov     [ebp-4], eax
0040101D      mov     ecx, [ebp-4]
00401020      push    ecx
00401021      push    offset aTotalD ; "total = %d\n"
00401026      call    printf
```

In [Example 7-5](#), `x` has been nicely labeled by IDA Pro Disassembler with the dummy name `var_4`. As we discussed in [Chapter 6](#), dummy names can be renamed to meaningful names that reflect their function. Having this local variable named `var_4` instead of `-4` simplifies your analysis, because once you rename `var_4` to `x`, you won't need to track the offset `-4` in your head throughout the

function.

Example 7-5. Assembly code for the local variable example shown in [Example 7-2](#), with labeling

```
00401006      mov     [ebp+var_4], 0
0040100D      mov     [ebp+var_8], 1
00401014      mov     eax, [ebp+var_4]
00401017      add     eax, [ebp+var_8]
0040101A      mov     [ebp+var_4], eax
0040101D      mov     ecx, [ebp+var_4]
00401020      push    ecx
00401021      push    offset aTotalD ; "total = %d\n"
00401026      call    printf
```

Disassembling Arithmetic Operations

Many different types of math operations can be performed in C programming, and we'll present the disassembly of those operations in this section.

Example 7-6 shows the C code for two variables and a variety of arithmetic operations. Two of these are the `--` and `++` operations, which are used to decrement by 1 and increment by 1, respectively. The `%` operation performs the modulo between the two variables, which is the remainder after performing a division operation.

Example 7-6. C code with two variables and a variety of arithmetic

```
int a = 0;
int b = 1;
a = a + 11;
a = a - b;
a--;
b++;
b = a % 3;
```

Example 7-7 shows the assembly for the C code shown in **Example 7-6**, which can be broken down to translate back to C.

Example 7-7. Assembly code for the arithmetic example in **Example 7-6**

00401006	mov	[ebp+var_4], 0	
0040100D	mov	[ebp+var_8], 1	
00401014	mov	eax, [ebp+var_4]	1
00401017	add	eax, 0Bh	
0040101A	mov	[ebp+var_4], eax	
0040101D	mov	ecx, [ebp+var_4]	
00401020	sub	ecx, [ebp+var_8]	2
00401023	mov	[ebp+var_4], ecx	
00401026	mov	edx, [ebp+var_4]	
00401029	sub	edx, 1	3
0040102C	mov	[ebp+var_4], edx	
0040102F	mov	eax, [ebp+var_8]	
00401032	add	eax, 1	4
00401035	mov	[ebp+var_8], eax	
00401038	mov	eax, [ebp+var_4]	
0040103B	cdq		
0040103C	mov	ecx, 3	
00401041	idiv	ecx	
00401043	mov	[ebp+var_8], edx	5

In this example, `a` and `b` are local variables because they are referenced by the stack. IDA Pro has labeled `a` as `var_4` and `b` as `var_8`. First, `var_4` and `var_8` are initialized to 0 and 1, respectively. `a` is moved into `eax` 1, and then 0x0b is added

to `eax`, thereby incrementing `a` by 11. `b` is then subtracted from `a` **2**. (The compiler decided to use the `sub` and `add` instructions **3** and **4**, instead of the `inc` and `dec` functions.)

The final five assembly instructions implement the modulo. When performing the `div` or `idiv` instruction **5**, you are dividing `edx:eax` by the operand and storing the result in `eax` and the remainder in `edx`. That is why `edx` is moved into `var_8` **5**.

Recognizing if Statements

Programmers use `if` statements to alter program execution based on certain conditions. `if` statements are common in C code and disassembly. We'll examine basic and nested `if` statements in this section. Your goal should be to learn how to recognize different types of `if` statements.

Example 7-8 displays a simple `if` statement in C with the assembly for this code shown in **Example 7-9**. Notice the conditional jump `jnz` at **2**. There must be a conditional jump for an `if` statement, but not all conditional jumps correspond to `if` statements.

Example 7-8. C code `if` statement example

```
int x = 1;
int y = 2;

if(x == y){
    printf("x equals y.\n");
}else{
    printf("x is not equal to y.\n");
}
```

Example 7-9. Assembly code for the `if` statement example in **Example 7-8**

```
00401006      mov     [ebp+var_8], 1
0040100D      mov     [ebp+var_4], 2
00401014      mov     eax, [ebp+var_8]
00401017      cmp     eax, [ebp+var_4] 1
0040101A      jnz     short loc_40102B 2
0040101C      push    offset aXEqualsY_ ; "x equals y.\n"
00401021      call    printf
00401026      add     esp, 4
00401029      jmp     short loc_401038 3
0040102B loc_40102B:
0040102B      push    offset aXIsNotEqualToY ; "x is not equal to y.\n"
00401030      call    printf
```

As you can see in **Example 7-9**, a decision must be made before the code inside the `if` statement in **Example 7-8** will execute. This decision corresponds to the conditional jump (`jnz`) shown at **2**. The decision to jump is made based on the comparison (`cmp`), which checks to see if `var_4` equals `var_8` (`var_4` and `var_8` correspond to `x` and `y` in our source code) at **1**. If the values are not equal, the jump occurs, and the code prints "x is not equal to y."; otherwise, the code continues the path of execution and prints "x equals y."

Notice also the jump (`jmp`) that jumps over the else section of the code at **3**. It is

important that you recognize that only one of these two code paths can be taken.

Analyzing Functions Graphically with IDA Pro

IDA Pro has a graphing tool that is useful in recognizing constructs, as shown in [Figure 7-1](#). This feature is the default view for analyzing functions.

[Figure 7-1](#) shows a graph of the assembly code example in [Example 7-9](#). As you can see, two different paths ([1](#) and [2](#)) of code execution lead to the end of the function, and each path prints a different string. Code path [1](#) will print "x equals y.", and [2](#) will print "x is not equal to y."

IDA Pro adds `false` [1](#) and `true` [2](#) labels at the decision points at the bottom of the upper code box. As you can imagine, graphing a function can greatly speed up the reverse-engineering process.

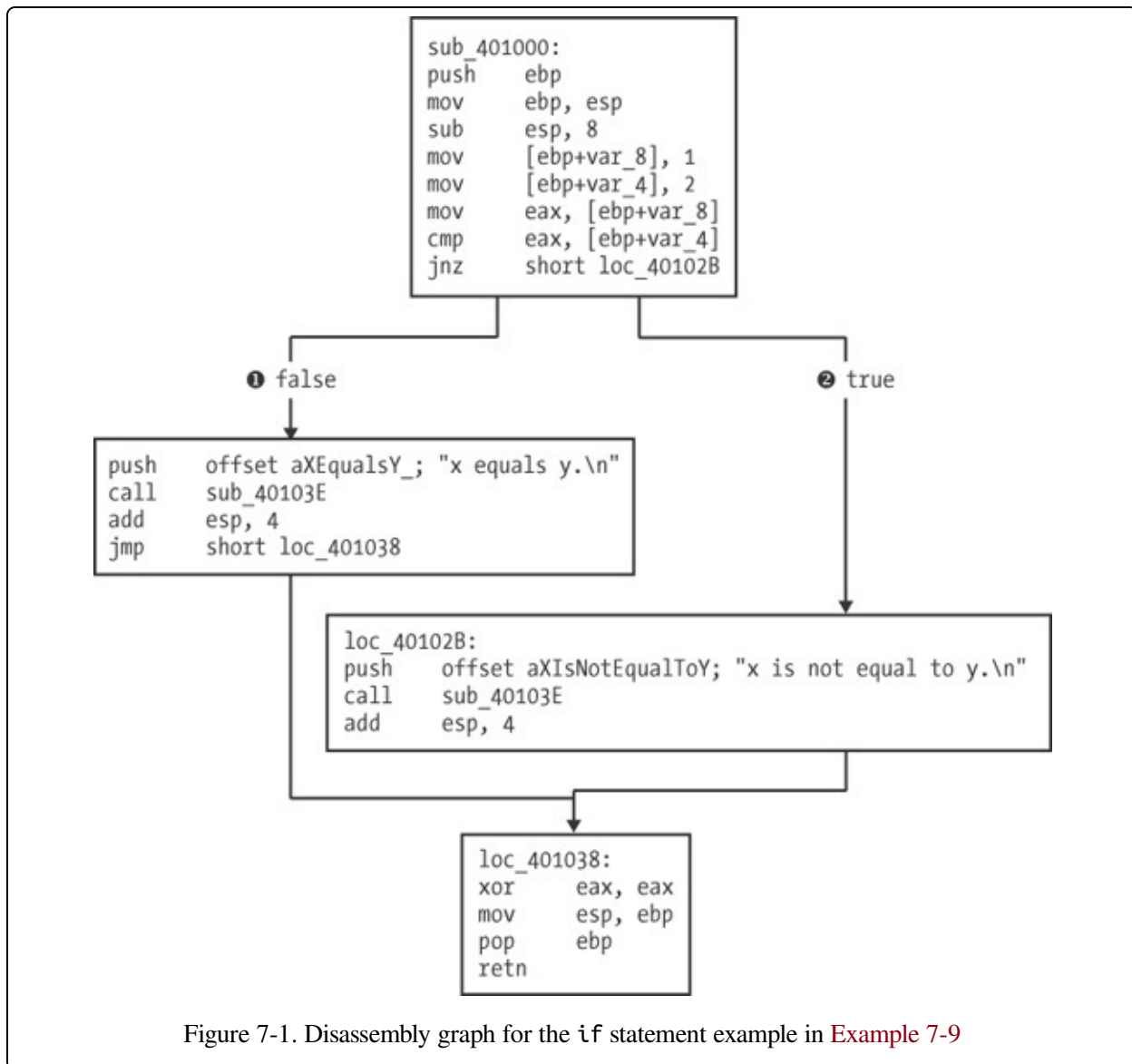
Recognizing Nested if Statements

[Example 7-10](#) shows C code for a nested `if` statement that is similar to [Example 7-8](#), except that two additional `if` statements have been added within the original `if` statement. These additional statements test to determine whether `z` is equal to 0.

Example 7-10. C code for a nested `if` statement

```
int x = 0;
int y = 1;
int z = 2;

if(x == y){
    if(z==0){
        printf("z is zero and x = y.\n");
    }else{
        printf("z is non-zero and x = y.\n");
    }
}else{
    if(z==0){
        printf("z zero and x != y.\n");
    }else{
        printf("z non-zero and x != y.\n");
    }
}
```

Despite this minor change to the C code, the assembly code is more complicated, as shown in Example 7-11.

Example 7-11. Assembly code for the nested `if` statement example shown in Example 7-10

```

00401006      mov     [ebp+var_8], 0
0040100D      mov     [ebp+var_4], 1
00401014      mov     [ebp+var_C], 2
0040101B      mov     eax, [ebp+var_8]
0040101E      cmp     eax, [ebp+var_4]
00401021      jnz     short loc_401047 1
00401023      cmp     [ebp+var_C], 0
00401027      jnz     short loc_401038 2
00401029      push    offset aZIsZeroAndXY_ ; "z is zero and x = y.\n"
0040102E      call    printf

```

```

00401033      add     esp, 4
00401036      jmp     short loc_401045
00401038 loc_401038:
00401038      push    offset aZIsNonZeroAndX ; "z is non-zero and x = y.\n"
0040103D      call    printf
00401042      add     esp, 4
00401045 loc_401045:
00401045      jmp     short loc_401069
00401047 loc_401047:
00401047      cmp     [ebp+var_C], 0
0040104B      jnz     short loc_40105C 3
0040104D      push    offset aZZeroAndXY_ ; "z zero and x != y.\n"
00401052      call    printf
00401057      add     esp, 4
0040105A      jmp     short loc_401069
0040105C loc_40105C:
0040105C      push    offset aZNonZeroAndXY_ ; "z non-zero and x != y.\n"
00401061      call    printf00401061

```

As you can see, three different conditional jumps occur. The first occurs if `var_4` does not equal `var_8` at **1**. The other two occur if `var_C` is not equal to zero at **2** and **3**.

Recognizing Loops

Loops and repetitive tasks are very common in all software, and it is important that you are able to recognize them.

Finding for Loops

The for loop is a basic looping mechanism used in C programming. for loops always have four components: initialization, comparison, execution instructions, and the increment or decrement.

Example 7-12 shows an example of a for loop.

Example 7-12. C code for a for loop

```
int i;

for(i=0; i<100; i++)
{
    printf("i equals %d\n", i);
}
```

In this example, the initialization sets `i` to 0 (zero), and the comparison checks to see if `i` is less than 100. If `i` is less than 100, the `printf` instruction will execute, the increment will add 1 to `i`, and the process will check to see if `i` is less than 100. These steps will repeat until `i` is greater than or equal to 100.

In assembly, the for loop can be recognized by locating the four components—initialization, comparison, execution instructions, and increment/decrement. For example, in **Example 7-13**, **1** corresponds to the initialization step. The code between **3** and **4** corresponds to the increment that is initially jumped over at **2** with a jump instruction. The comparison occurs at **5**, and at **6**, the decision is made by the conditional jump. If the jump is not taken, the `printf` instruction will execute, and an unconditional jump occurs at **7**, which causes the increment to occur.

Example 7-13. Assembly code for the for loop example in **Example 7-12**

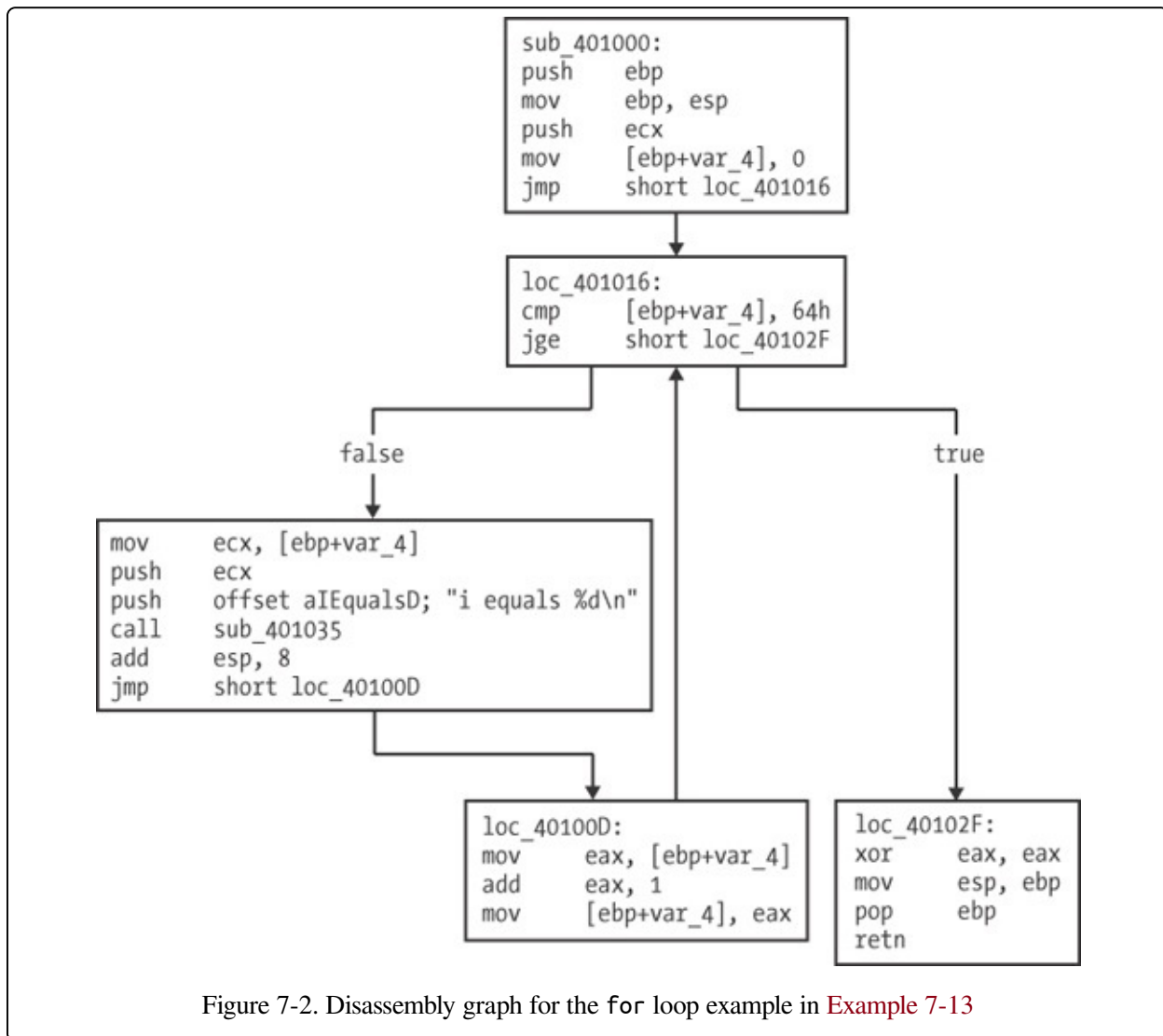
```
00401004      mov     [ebp+var_4], 0 1
0040100B      jmp     short loc_401016 2
0040100D loc_40100D:
0040100D      mov     eax, [ebp+var_4] 3
00401010      add     eax, 1
00401013      mov     [ebp+var_4], eax 4
00401016 loc_401016:
```

```

00401016    cmp     [ebp+var_4], 64h 5
0040101A    jge     short loc_40102F 6
0040101C    mov     ecx, [ebp+var_4]
0040101F    push    ecx
00401020    push    offset aID ; "i equals %d\n"
00401025    call    printf
0040102A    add     esp, 8
0040102D    jmp     short loc_40100D 7

```

A for loop can be recognized using IDA Pro's graphing mode, as shown in [Figure 7-2](#).



In the figure, the upward pointing arrow after the increment code indicates a loop. These arrows make loops easier to recognize in the graph view than in the standard disassembly view. The graph displays five boxes: The top four are the components of the for loop (initialization, comparison, execution, and increment, in that

order). The box on the bottom right is the function epilogue, which we described in [Chapter 5](#) as the portion of a function responsible for cleaning up the stack and returning.

Finding while Loops

The `while` loop is frequently used by malware authors to loop until a condition is met, such as receiving a packet or command. `while` loops look similar to `for` loops in assembly, but they are easier to understand. The `while` loop in [Example 7-14](#) will continue to loop until the status returned from `checkResult` is 0.

Example 7-14. C code for a `while` loop

```
int status=0;
int result = 0;

while(status == 0){
    result = performAction();
    status = checkResult(result);
}
```

The assembly code in [Example 7-15](#) looks similar to the `for` loop, except that it lacks an increment section. A conditional jump occurs at **1** and an unconditional jump at **2**, but the only way for this code to stop executing repeatedly is for that conditional jump to occur.

Example 7-15. Assembly code for the `while` loop example in [Example 7-14](#)

```
00401036      mov     [ebp+var_4], 0
0040103D      mov     [ebp+var_8], 0
00401044 loc_401044:
00401044      cmp     [ebp+var_4], 0
00401048      jnz     short loc_401063 1
0040104A      call    performAction
0040104F      mov     [ebp+var_8], eax
00401052      mov     eax, [ebp+var_8]
00401055      push    eax
00401056      call    checkResult
0040105B      add     esp, 4
0040105E      mov     [ebp+var_4], eax
00401061      jmp     short loc_401044 2
```

Understanding Function Call Conventions

In [Chapter 5](#), we discussed how the stack and the `call` instruction are used for function calls. Function calls can appear differently in assembly code, and calling conventions govern the way the function call occurs. These conventions include the order in which parameters are placed on the stack or in registers, and whether the caller or the function called (the callee) is responsible for cleaning up the stack when the function is complete.

The calling convention used depends on the compiler, among other factors. There are often subtle differences in how compilers implement these conventions, so it can be difficult to interface code that is compiled by different compilers. However, you need to follow certain conventions when using the Windows API, and these are uniformly implemented for compatibility (as discussed in [Chapter 8](#)).

We will use the pseudocode in [Example 7-16](#) to describe each of the calling conventions.

Example 7-16. Pseudocode for a function call

```
int test(int x, int y, int z);
int a, b, c, ret;

ret = test(a, b, c);
```

The three most common calling conventions you will encounter are `cdecl`, `stdcall`, and `fastcall`. We discuss the key differences between them in the following sections.

NOTE

Although the same conventions can be implemented differently between compilers, we'll focus on the most common ways they are used.

`cdecl`

`cdecl` is one of the most popular conventions and was described in [Chapter 5](#) when we introduced the stack and function calls. In `cdecl`, parameters are pushed onto the stack from right to left, the caller cleans up the stack when the function is complete, and the return value is stored in EAX. [Example 7-17](#) shows an example

of what the disassembly would look like if the code in [Example 7-16](#) were compiled to use `cdecl`.

Example 7-17. `cdecl` function call

```
push c
push b
push a
call test
add esp, 12
mov ret, eax
```

Notice in the highlighted portion that the stack is cleaned up by the caller. In this example, the parameters are pushed onto the stack from right to left, beginning with `c`.

stdcall

The popular `stdcall` convention is similar to `cdecl`, except `stdcall` requires the callee to clean up the stack when the function is complete. Therefore, the `add` instruction highlighted in [Example 7-17](#) would not be needed if the `stdcall` convention were used, since the function called would be responsible for cleaning up the stack.

The `test` function in [Example 7-16](#) would be compiled differently under `stdcall`, because it must be concerned with cleaning up the stack. Its epilogue would need to take care of the cleanup.

`stdcall` is the standard calling convention for the Windows API. Any code calling these API functions will not need to clean up the stack, since that's the responsibility of the DLLs that implement the code for the API function.

fastcall

The `fastcall` calling convention varies the most across compilers, but it generally works similarly in all cases. In `fastcall`, the first few arguments (typically two) are passed in registers, with the most commonly used registers being `EDX` and `ECX` (the Microsoft `fastcall` convention). Additional arguments are loaded from right to left, and the calling function is usually responsible for cleaning up the stack, if necessary. It is often more efficient to use `fastcall` than other conventions, because the code doesn't need to involve the stack as much.

Push vs. Move

In addition to using the different calling conventions described so far, compilers may also choose to use different instructions to perform the same operation, usually when the compiler decides to move rather than push things onto the stack.

Example 7-18 shows a C code example of a function call. The function `adder` adds two arguments and returns the result. The `main` function calls `adder` and prints the result using `printf`.

Example 7-18. C code for a function call

```
int adder(int a, int b)
{
    return a+b;
}

void main()
{
    int x = 1;
    int y = 2;

    printf("the function returned the number %d\n", adder(x,y));
}
```

The assembly code for the `adder` function is consistent across compilers and is displayed in **Example 7-19**. As you can see, this code adds `arg_0` to `arg_4` and stores the result in `EAX`. (As discussed in **Chapter 5**, `EAX` stores the return value.)

Example 7-19. Assembly code for the `adder` function in **Example 7-18**

00401730	push	ebp
00401731	mov	ebp, esp
00401733	mov	eax, [ebp+arg_0]
00401736	add	eax, [ebp+arg_4]
00401739	pop	ebp
0040173A	retn	

Table 7-1 displays different calling conventions used by two different compilers: Microsoft Visual Studio and GNU Compiler Collection (GCC). On the left, the parameters for `adder` and `printf` are pushed onto the stack before the call. On the right, the parameters are moved onto the stack before the call. You should be prepared for both types of calling conventions, because as an analyst, you won't have control over the compiler. For example, one instruction on the left does not correspond to any instruction on the right. This instruction restores the stack pointer, which is not necessary on the right because the stack pointer is never altered.

NOTE

Remember that even when the same compiler is used, there can be differences in calling conventions depending on the various settings and options.

Table 7-1. Assembly Code for a Function Call with Two Different Calling Conventions

Visual Studio version			GCC version		
00401746	mov	[ebp+var_4], 1	00401085	mov	[ebp+var_4], 1
0040174D	mov	[ebp+var_8], 2	0040108C	mov	[ebp+var_8], 2
00401754	mov	eax, [ebp+var_8]	00401093	mov	eax, [ebp+var_8]
00401757	push	eax	00401096	mov	[esp+4], eax
00401758	mov	ecx, [ebp+var_4]	0040109A	mov	eax, [ebp+var_4]
0040175B	push	ecx	0040109D	mov	[esp], eax
0040175C	call	adder	004010A0	call	adder
00401761	add	esp, 8	004010A5	mov	[esp+4], eax
00401764	push	eax	004010A9	mov	[esp], offset
00401765	push	offset	TheFunctionRet		
TheFunctionRet			004010B0	call	printf
0040176A	call	ds:printf			

Analyzing switch Statements

switch statements are used by programmers (and malware authors) to make a decision based on a character or integer. For example, backdoors commonly select from a series of actions using a single byte value. switch statements are compiled in two common ways: using the if style or using jump tables.

If Style

Example 7-20 shows a simple switch statement that uses the variable `i`. Depending on the value of `i`, the code under the corresponding case value will be executed.

Example 7-20. C code for a three-option switch statement

```
switch(i)
{
    case 1:
        printf("i = %d", i+1);
        break;
    case 2:
        printf("i = %d", i+2);
        break;
    case 3:
        printf("i = %d", i+3);
        break;
    default:
        break;
}
```

This switch statement has been compiled into the assembly code shown in **Example 7-21**. It contains a series of conditional jumps between **1** and **2**. The conditional jump determination is made by the comparison that occurs directly before each jump.

The switch statement has three options, shown at **3**, **4**, and **5**. These code sections are independent of each other because of the unconditional jumps to the end of the listing. (You'll probably find that switch statements are easier to understand using the graph shown in **Figure 7-3**.)

Example 7-21. Assembly code for the switch statement example in **Example 7-20**

```
00401013      cmp     [ebp+var_8], 1
00401017      jz      short loc_401027 1
00401019      cmp     [ebp+var_8], 2
0040101D      jz      short loc_40103D
0040101F      cmp     [ebp+var_8], 3
```

```

00401023      jz      short loc_401053
00401025      jmp     short loc_401067 ❷
00401027 loc_401027:
00401027      mov     ecx, [ebp+var_4] ❸
0040102A      add     ecx, 1
0040102D      push    ecx
0040102E      push    offset unk_40C000 ; i = %d
00401033      call    printf
00401038      add     esp, 8
0040103B      jmp     short loc_401067
0040103D loc_40103D:
0040103D      mov     edx, [ebp+var_4] ❹
00401040      add     edx, 2
00401043      push    edx
00401044      push    offset unk_40C004 ; i = %d
00401049      call    printf
0040104E      add     esp, 8
00401051      jmp     short loc_401067
00401053 loc_401053:
00401053      mov     eax, [ebp+var_4] ❺
00401056      add     eax, 3
00401059      push    eax
0040105A      push    offset unk_40C008 ; i = %d
0040105F      call    printf
00401064      add     esp, 8

```

Figure 7-3 breaks down each of the switch options by splitting up the code to be executed from the next decision to be made. Three of the boxes in the figure, labeled ❶, ❷, and ❸, correspond directly to the case statement's three different options. Notice that all of these boxes terminate at the bottom box, which is the end of the function. You should be able to use this graph to see the three checks the code must go through when `var_8` is greater than 3.

From this disassembly, it is difficult, if not impossible, to know whether the original code was a `switch` statement or a sequence of `if` statements, because a compiled `switch` statement looks like a group of `if` statements—both can contain a bunch of `cmp` and `Jcc` instructions. When performing your disassembly, you may not always be able to get back to the original source code, because there may be multiple ways to represent the same code constructs in assembly, all of which are valid and equivalent.

Jump Table

The next disassembly example is commonly found with large, contiguous `switch` statements. The compiler optimizes the code to avoid needing to make so many comparisons. For example, if in **Example 7-20** the value of `i` were 3, three different comparisons would take place before the third case was executed. In

Example 7-22, we add one case to **Example 7-20** (as you can see by comparing the listings), but the assembly code generated is drastically different.

Example 7-22. C code for a four-option switch statement

```
switch(i)
{
    case 1:
        printf("i = %d", i+1);
        break;
    case 2:
        printf("i = %d", i+2);
        break;
    case 3:
        printf("i = %d", i+3);
        break;
    case 4:
        printf("i = %d", i+3);
        break;
    default:
        break;
}
```

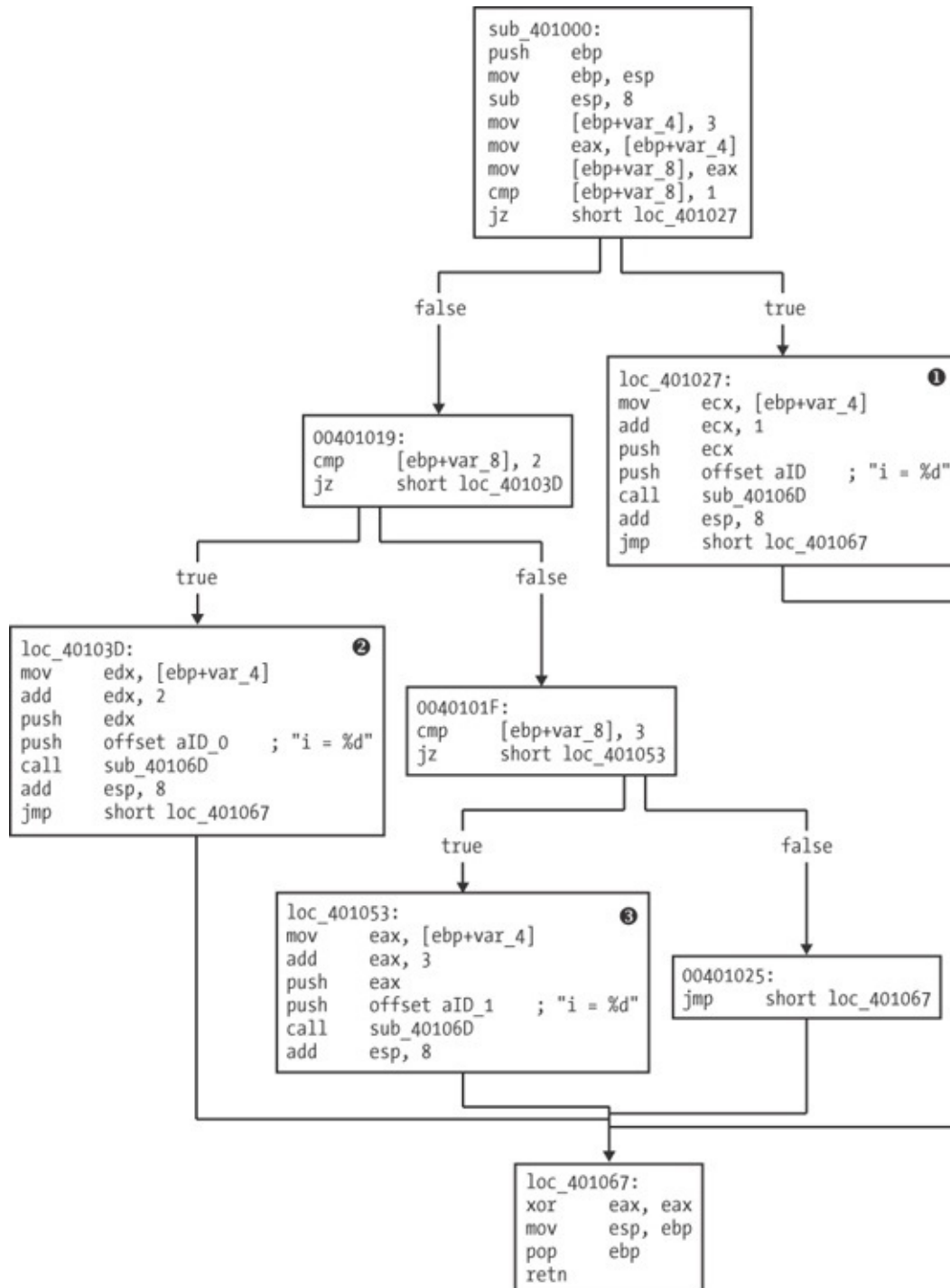


Figure 7-3. Disassembly graph of the if style switch statement example in [Example 7-21](#)

The more efficient assembly code in [Example 7-23](#) uses a jump table, shown at [2](#), which defines offsets to additional memory locations. The switch variable is used as an index into the jump table.

In this example, `ecx` contains the switch variable, and 1 is subtracted from it in the first line. In the C code, the switch table range is 1 through 4, and the assembly code must adjust it to 0 through 3 so that the jump table can be properly indexed. The jump instruction at **1** is where the target is based on the jump table.

In this jump instruction, `edx` is multiplied by 4 and added to the base of the jump table (0x401088) to determine which case code block to jump to. It is multiplied by 4 because each entry in the jump table is an address that is 4 bytes in size.

Example 7-23. Assembly code for the switch statement example in [Example 7-22](#)

```

00401016      sub     ecx, 1
00401019      mov     [ebp+var_8], ecx
0040101C      cmp     [ebp+var_8], 3
00401020      ja      short loc_401082
00401022      mov     edx, [ebp+var_8]
00401025      jmp     ds:off_401088[edx*4] 1
0040102C      loc_40102C:
                ...
00401040      jmp     short loc_401082
00401042      loc_401042:
                ...
00401056      jmp     short loc_401082
00401058      loc_401058:
                ...
0040106C      jmp     short loc_401082
0040106E      loc_40106E:
                ...
00401082      loc_401082:
00401082      xor     eax, eax
00401084      mov     esp, ebp
00401086      pop     ebp
00401087      retn
00401087      _main  endp
00401088      2off_401088 dd offset loc_40102C
0040108C      dd offset loc_401042
00401090      dd offset loc_401058
00401094      dd offset loc_40106E

```

The graph in [Figure 7-4](#) for this type of switch statement is clearer than the standard disassembly view.

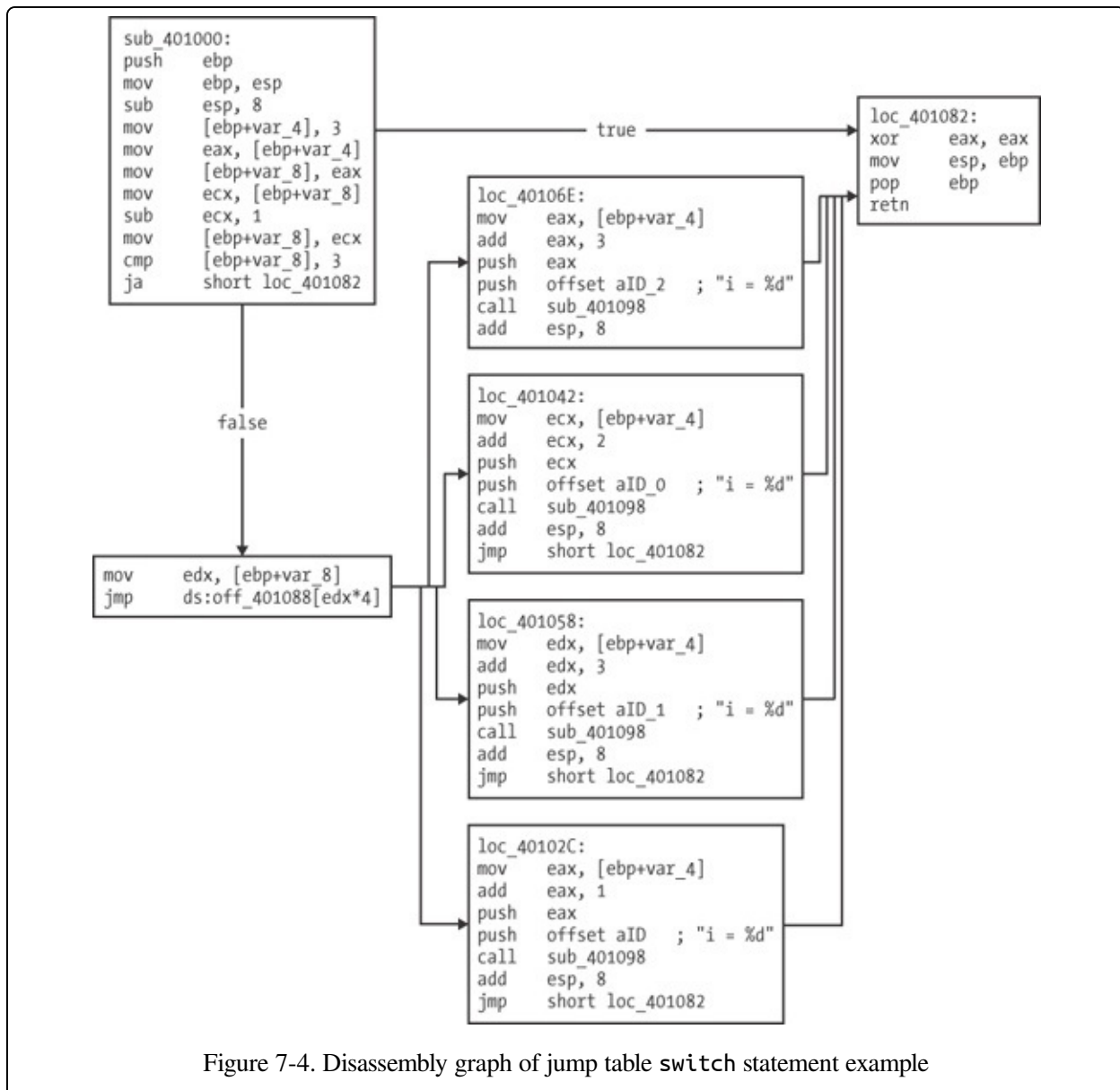


Figure 7-4. Disassembly graph of jump table switch statement example

As you can see, each of the four cases is broken down clearly into separate assembly code chunks. These chunks appear one after another in a column after the jump table determines which one to use. Notice that all of these boxes and the initial box terminate at the right box, which is the end of the function.

Disassembling Arrays

Arrays are used by programmers to define an ordered set of similar data items. Malware sometimes uses an array of pointers to strings that contain multiple hostnames that are used as options for connections.

Example 7-24 shows two arrays used by one program, both of which are set during the iteration through the for loop. Array a is locally defined, and array b is globally defined. These definitions will impact the assembly code.

Example 7-24. C code for an array

```
int b[5] = {123,87,487,7,978};
void main()
{
    int i;
    int a[5];

    for(i = 0; i<5; i++)
    {
        a[i] = i;
        b[i] = i;
    }
}
```

In assembly, arrays are accessed using a base address as a starting point. The size of each element is not always obvious, but it can be determined by seeing how the array is being indexed. **Example 7-25** shows the assembly code for **Example 7-24**.

Example 7-25. Assembly code for the array in **Example 7-24**

```
00401006      mov     [ebp+var_18], 0
0040100D      jmp     short loc_401018
0040100F loc_40100F:
0040100F      mov     eax, [ebp+var_18]
00401012      add     eax, 1
00401015      mov     [ebp+var_18], eax
00401018 loc_401018:
00401018      cmp     [ebp+var_18], 5
0040101C      jge     short loc_401037
0040101E      mov     ecx, [ebp+var_18]
00401021      mov     edx, [ebp+var_18]
00401024      mov     [ebp+ecx*4+var_14], edx 1
00401028      mov     eax, [ebp+var_18]
0040102B      mov     ecx, [ebp+var_18]
0040102E      mov     dword_40A000[ecx*4], eax 2
00401035      jmp     short loc_40100F
```

In this listing, the base address of array b corresponds to dword_40A000, and the base address of array a corresponds to var_14. Since these are both arrays of

integers, each element is of size 4, although the instructions at **1** and **2** differ for accessing the two arrays. In both cases, `ecx` is used as the index, which is multiplied by 4 to account for the size of the elements. The resulting value is added to the base address of the array to access the proper array element.

Identifying Structs

Structures (or structs, for short) are similar to arrays, but they comprise elements of different types. Structures are commonly used by malware authors to group information. It's sometimes easier to use a structure than to maintain many different variables independently, especially if many functions need access to the same group of variables. (Windows API functions often use structures that must be created and maintained by the calling program.) In [Example 7-26](#), we define a structure at [1](#) made up of an integer array, a character, and a double. In `main`, we allocate memory for the structure and pass the struct to the `test` function. The `struct gms` defined at [2](#) is a global variable.

Example 7-26. C code for a struct example

```
struct my_structure { 1
    int x[5];
    char y;
    double z;
};

struct my_structure *gms; 2

void test(struct my_structure *q)
{
    int i;
    q->y = 'a';
    q->z = 15.6;
    for(i = 0; i<5; i++){
        q->x[i] = i;
    }
}

void main()
{
    gms = (struct my_structure *) malloc(
        sizeof(struct my_structure));
    test(gms);
}
```

Structures (like arrays) are accessed with a base address used as a starting pointer. It is difficult to determine whether nearby data types are part of the same struct or whether they just happen to be next to each other. Depending on the structure's context, your ability to identify a structure can have a significant impact on your ability to analyze malware.

[Example 7-27](#) shows the `main` function from [Example 7-26](#), disassembled. Since

the struct `gms` is a global variable, its base address will be the memory location `dword_40EA30` as shown in [Example 7-27](#). The base address of this structure is passed to the `sub_401000` (test) function via the `push eax` at [1](#).

Example 7-27. Assembly code for the `main` function in the struct example in [Example 7-26](#)

```
00401050      push     ebp
00401051      mov      ebp, esp
00401053      push     20h
00401055      call     malloc
0040105A      add      esp, 4
0040105D      mov      dword_40EA30, eax
00401062      mov      eax, dword_40EA30
00401067      push     eax 1
00401068      call     sub_401000
0040106D      add      esp, 4
00401070      xor      eax, eax
00401072      pop      ebp
00401073      retn
```

[Example 7-28](#) shows the disassembly of the `test` method shown in [Example 7-26](#). `arg_0` is the base address of the structure. Offset `0x14` stores the character within the struct, and `0x61` corresponds to the letter `a` in ASCII.

Example 7-28. Assembly code for the `test` function in the struct example in [Example 7-26](#)

```
00401000      push     ebp
00401001      mov      ebp, esp
00401003      push     ecx
00401004      mov      eax, [ebp+arg_0]
00401007      mov      byte ptr [eax+14h], 61h
0040100B      mov      ecx, [ebp+arg_0]
0040100E      fld      ds:dbl_40B120 1
00401014      fstp     qword ptr [ecx+18h]
00401017      mov      [ebp+var_4], 0
0040101E      jmp      short loc_401029
00401020 loc_401020:
00401020      mov      edx, [ebp+var_4]
00401023      add      edx, 1
00401026      mov      [ebp+var_4], edx
00401029 loc_401029:
00401029      cmp      [ebp+var_4], 5
0040102D      jge      short loc_40103D
0040102F      mov      eax, [ebp+var_4]
00401032      mov      ecx, [ebp+arg_0]
00401035      mov      edx, [ebp+var_4]
00401038      mov      [ecx+eax*4], edx 2
0040103B      jmp      short loc_401020
0040103D loc_40103D:
0040103D      mov      esp, ebp
0040103F      pop      ebp
00401040      retn
```

We can tell that offset 0x18 is a double because it is used as part of a floating-point instruction at **1**. We can also tell that integers are moved into offset 0, 4, 8, 0xC, and 0x10 by examining the `for` loop and where these offsets are accessed at **2**. We can infer the contents of the structure from this analysis.

In IDA Pro, you can create structures and assign them to memory references using the T hotkey. Doing this will change the instruction `mov [eax+14h], 61h` to `mov [eax + my_structure.y], 61h`. The latter is easier to read, and marking structures can often help you understand the disassembly more quickly, especially if you are constantly viewing the structure used. To use the T hotkey effectively in this example, you would need to create the `my_structure` structure manually using IDA Pro's structure window. This can be a tedious process, but it can be helpful for structures that you encounter frequently.

Analyzing Linked List Traversal

A linked list is a data structure that consists of a sequence of data records, and each record includes a field that contains a reference (link) to the next record in the sequence. The principal benefit of using a linked list over an array is that the order of the linked items can differ from the order in which the data items are stored in memory or on disk. Therefore, linked lists allow the insertion and removal of nodes at any point in the list.

Example 7-29 shows a C code example of a linked list and its traversal. This linked list consists of a series of node structures named `pnode`, and it is manipulated with two loops. The first loop at **1** creates 10 nodes and fills them with data. The second loop at **2** iterates over all the records and prints their contents.

Example 7-29. C code for a linked list traversal

```
struct node
{
    int x;
    struct node next;
};

typedef struct node pnode;

void main()
{
    pnode curr, * head;
    int i;

    head = NULL;

    for(i=1;i<=10;i++) 1
    {
        curr = (pnode *)malloc(sizeof(pnode));
        curr->x = i;
        curr->next = head;
        head = curr;
    }

    curr = head;

    while(curr) 2
    {
        printf("%d\n", curr->x);
        curr = curr->next ;
    }
}
```

The best way to understand the disassembly is to identify the two code constructs

within the `main` method. And that is, of course, the crux of this chapter: Your ability to recognize these constructs makes the analysis easier.

In [Example 7-30](#), we identify the `for` loop first. `var_C` corresponds to `i`, which is the counter for the loop. `var_8` corresponds to the `head` variable, and `var_4` is the `curr` variable. `var_4` is a pointer to a struct with two variables that are assigned values (shown at [1](#) and [2](#)).

The `while` loop ([3](#) through [5](#)) executes the iteration through the linked list. Within the loop, `var_4` is set to the next record in the list at [4](#).

Example 7-30. Assembly code for the linked list traversal example in [Example 7-29](#)

```

0040106A      mov     [ebp+var_8], 0
00401071      mov     [ebp+var_C], 1
00401078
00401078 loc_401078:
00401078      cmp     [ebp+var_C], 0Ah
0040107C      jg      short loc_4010AB
0040107E      mov     [esp+18h+var_18], 8
00401085      call    malloc
0040108A      mov     [ebp+var_4], eax
0040108D      mov     edx, [ebp+var_4]
00401090      mov     eax, [ebp+var_C]
00401093      mov     [edx], eax 1
00401095      mov     edx, [ebp+var_4]
00401098      mov     eax, [ebp+var_8]
0040109B      mov     [edx+4], eax 2
0040109E      mov     eax, [ebp+var_4]
004010A1      mov     [ebp+var_8], eax
004010A4      lea     eax, [ebp+var_C]
004010A7      inc     dword ptr [eax]
004010A9      jmp     short loc_401078
004010AB loc_4010AB:
004010AB      mov     eax, [ebp+var_8]
004010AE      mov     [ebp+var_4], eax
004010B1
004010B1 loc_4010B1:
004010B1      cmp     [ebp+var_4], 0 3
004010B5      jz      short locret_4010D7
004010B7      mov     eax, [ebp+var_4]
004010BA      mov     eax, [eax]
004010BC      mov     [esp+18h+var_14], eax
004010C0      mov     [esp+18h+var_18], offset aD ; "%d\n"
004010C7      call    printf
004010CC      mov     eax, [ebp+var_4]
004010CF      mov     eax, [eax+4]
004010D2      mov     [ebp+var_4], eax 4
004010D5      jmp     short loc_4010B1 5

```

To recognize a linked list, you must first recognize that some object contains a pointer that points to another object of the same type. The recursive nature of the

objects is what makes it linked, and this is what you need to recognize from the disassembly.

In this example, realize that at `4`, `var_4` is assigned `eax`, which comes from `[eax+4]`, which itself came from a previous assignment of `var_4`. This means that whatever struct `var_4` is must contain a pointer 4 bytes into it. This points to another struct that must also contain a pointer 4 bytes into another struct, and so on.

Conclusion

This chapter was designed to expose you to a constant task in malware analysis: abstracting yourself from the details. Don't get bogged down in the low-level details, but develop the ability to recognize what the code is doing at a higher level.

We've shown you each of the major C coding constructs in both C and assembly to help you quickly recognize the most common constructs during analysis. We've also offered a couple of examples showing where the compiler decided to do something different, in the case of structs and (when an entirely different compiler was used) in the case of function calls. Developing this insight will help you as you navigate the path toward recognizing new constructs when you encounter them in the wild.